RESEARCH AND REFACTORING NOTE

Further simplifications for the DiophantineConsistency Lean development

After assuming the previous 22 proposal packets have been applied

Prepared for Vladimir Reshetnikov 26 October 2026

Abstract

Five further directions survive a comparison with the current research collection and incoming index. They comprise six concrete mechanisms: reuse five hidden Jones evaluation lemmas; replace an auxiliary norm bound by a positive polynomial gap; share an affine domination bound between carrier soundness and completeness; derive the helical row theorem from a one-dimensional periodic function; interpret monomial lists through ordinary list sums; and eliminate function-valued witnesses whose values are already fixed.

The supplied patches remove **94 physical Lean source lines across eight files**, including the new helper and import costs. Three complete replacement modules were compiled. The other changes were checked through exact theorem bodies or a harness using the actual source definitions, with static checks binding those kernels to the proposed files. No complete project or downstream dependency rebuild is claimed. Existing executable definitions and public theorem statements are preserved; five previously private theorem names and one new helper become available to clients. The packet contains the source, PDF, full replacement files, patches, novelty ledger, compiler logs, source hashes, and replay scripts.

Direction	Lines removed	Strongest completed check
Jones evaluation API reuse	49	Actual-definition harness; full-file static comparison
Carrier norm gap and affine domination	16	Exact three replacement theorems and helper compiled
Helical orbit periodicity	10	Exact theorem and unique factorization compiled
Monomial-list sum interface	9	Complete replacement module compiled
Deterministic function witnesses	10	Both complete modules; whole-record `rfl` checks
Total	**94**	**19 lines in complete-module checks; 75 in narrower checks**

Repository snapshot: `efa64ca941cdae37bb6f7fc5093ae8bb69296ca3`.

Toolchain: Lean 4.32.0; mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`.

These are proposal artifacts against a pinned snapshot, with explicitly delimited validation.

Contents

1 Scope, baseline, and what counts as additional **3**

1.1 The counterfactual baseline is semantic . . . 3

1.2 What is measured and preserved . . . 3

2 Reuse the five hidden Jones evaluation lemmas **4**

2.1 The remaining duplication . . . 4

2.2 Proposed change . . . 4

2.3 Why this remains new and useful . . . 5

2.4 Validation boundary . . . 5

3 Two further reductions in the thirteen-coordinate carrier **5**

3.1 A positive gap replaces the auxiliary norm contradiction . . . 5

3.2 One affine domination theorem serves both directions . . . 6

3.3 Boundary cases and measured cost . . . 7

4 Replace the helical integer induction by periodicity **8**

5 Use one list-sum bridge for executable monomial semantics **8**

5.1 Why the list representation should remain . . . 9

5.2 Increment and validation . . . 9

6 Eliminate function witnesses that are already determined **10**

7 Validation, accounting, and a practical adoption order **10**

7.1 Evidence has several scopes . . . 10

7.2 Exact file-level accounting . . . 11

7.3 Suggested adoption order . . . 12

8 Ideas deliberately excluded during this iteration **12**

9 Archive guide and source references **12**

1 Scope, baseline, and what counts as additional

The question is about work remaining after earlier simplifications, including proposals that have not yet been integrated. Accordingly, the adopted/pending distinction in the repository was not used to decide novelty. A pending idea was excluded just as an adopted one was. For example, the large existing proposal for `NativeCoefficientBounds` was not presented again merely because its shorter implementation is still pending in the maintained tree.

The review inspected the 470 top-level maintained Lean modules as an inventory, read the relevant proof blocks and their dependencies, and reviewed all 22 placed simplification packets `2026-10-26a` through `2026-10-26v`. The novelty inventory contains 637 text, Lean, and patch files, totaling 6,484,226 bytes. This is a corpus inventory, not a claim that every line of every implementation was manually reread. Main reports, relevant supplementary notes, existing audit ledgers, and targeted full-corpus searches were used together.

At this snapshot, `docs/incoming` contains a README rather than the retired archive bytes. Its current entries map the 22 Diophantine proof packets into the research directory. The complete index and the relevant retirement summaries were reviewed, including summaries of older K215 work. No claim is made to have recursively re-audited all historical material linked from unrelated projects. This boundary is recorded in `evidence/novelty_audit.md` and the independent review.

The source snapshot was authenticated independently of the initial download: GitHub tree metadata at the pinned commit was compared with the Git blob hashes of 1,763 retained files from the Lean tree and proposal packets. There were no mismatches. The toolchain and mathlib revisions agree with the repository's `lean-toolchain` and `lake-manifest.json`. The detailed source manifests and the smaller authentication receipt accompany the report.

1.1 The counterfactual baseline is semantic

Some previous packets offer competing implementations or sketches of the same interface. Treating them as applied means assuming their promised proof obligations and usable interfaces are already available. It does not mean pretending that every historical patch can be applied simultaneously without adaptation.

The distinction matters here. Earlier assignment-naturality and finite-witness splitting work is assumed finished before the final function-witness substitution in Section 6. Earlier integer-unit work receives no credit in the carrier example: the supplied patch preserves the original nonzero-ordinate proof verbatim and replaces only the remaining root-bound tail. Earlier coefficient-table naturality does not remove the separate list-interpreter proofs treated in Section 5.

The literal patches are against the observed commit. Their claimed novelty concerns the residual obligations after the earlier interfaces are granted. This report does not claim to have built a hypothetical checkout containing all competing older proposals.

1.2 What is measured and preserved

The line counts are whole-file physical line counts, not estimated percentages of the repository or reductions in proof-term size. Helpers, imports, comments, and blank lines are included. The affected files contain 1,401 lines before the patch set and 1,307 afterward. The larger benefit in several cases is that one mathematical obligation has one proof and one place to change.

The proposals preserve existing executable definitions, expression trees, coordinate maps, polynomial definitions, witness layouts, and public theorem conclusions. They do not lower the number of unknowns in a Diophantine equation. The two composition changes have particularly strong evidence: the original complete constructors were reconstructed and proved definitionally equal to the replacements by `rfl`. No timing improvement is inferred from the source reductions.

2 Reuse the five hidden Jones evaluation lemmas

2.1 The remaining duplication

`JonesSyntax.eval_rad` and its four companion lemmas already show that evaluating a Jones syntax field agrees with the corresponding field of the evaluated integer record. However, those lemmas are private. `ExecutableJones` imports `JonesSyntax` and then proves the same five facts again before its own `ExecutableJones.eval_withInput` theorem.

The duplicated proof is not merely a short cast. The radicand proof unfolds a substantial collection of named intermediate expressions, and the divisor, dividend, and margin proofs each list their own dependencies. A change to a common field can therefore require editing two long, nearly parallel unfolding lists.

Existing private theorem	Mathematical field transported by evaluation
`eval_rad`	Each of the two radicands
`eval_wt`	The corresponding radical weight
`eval_dvsr`	Divisor expression
`eval_dvnd`	Dividend expression
`eval_marg`	Margin expression

For a field operation F, the common content is

$$\mathrm{eval}_a(F(v)) = F\big(\mathrm{evalExprVars}_a(v)\big).$$

The right-hand F acts on the existing integer record; the left-hand F builds arithmetic syntax. The source already contains the exact bridge needed by the executable client.

2.2 Proposed change

Remove `private` from the five theorems in `JonesSyntax`. Delete their duplicate block in `ExecutableJones`. In the executable client's existing `simp only` call, use the qualified shared names:

```
JonesSyntax.eval_rad, JonesSyntax.eval_wt,
JonesSyntax.eval_dvsr, JonesSyntax.eval_dvnd,
JonesSyntax.eval_marg
```

Do not add global simplifier attributes. The existing explicit rewrite boundary is appropriate: it lets the client choose when to cross from syntax to the semantic record, while keeping the large field expansions behind named theorems. No new dependency edge is needed because the client already imports the provider.

Some existing statements write evaluation as `(fun e => e.eval a) v.dvsr`, while the duplicate writes `v.dvsr.eval a`. These are definitionally equal by beta reduction. The difference does not call for an adapter theorem or a stronger hypothesis.

2.3 Why this remains new and useful

Earlier proposals covered coefficient-row homomorphisms, shared scalar tails, and moving degree or renaming lemmas to earlier modules. They do not provide these five hidden `RingVars` evaluation lemmas to the executable Jones client. This proposal closes that specific remaining API gap.

The implementation saves 49 lines: `JonesSyntax` remains 179 lines and `ExecutableJones` changes from 300 to 251. It also makes future changes to the five field semantics local to the shared proof. The syntax constructors, the executable coefficient extraction, and both final equation expressions remain untouched.

2.4 Validation boundary

The supplied harness contains the actual pinned `RingVars`, integer `Vars`, and evaluation definitions used by these statements. It checks all five client-shaped statements through the shared proofs. Source hashes identify the extracted definitions, including the two HilbertTenthProblem dependencies. The candidate files and patch were independently compared with the originals.

The complete `JonesSyntax` and `ExecutableJones` modules, and the complete `eval_withInput` client in its original dependency graph, were not rebuilt in this session. Thus this is a checked semantic reuse proposal with source-level compatibility evidence, awaiting a full integration build. That is a narrower claim than a completed module refactor.

3 Two further reductions in the thirteen-coordinate carrier

The next two changes concern scalar obligations in `ThirteenCarrierSoundness` and `ThirteenFreshSeed`. They use different mathematical observations, but belong in one carrier section because the same soundness/completeness boundary benefits from both.

3.1 A positive gap replaces the auxiliary norm contradiction

The existing `ThirteenCarrierSoundness.auxiliary_estimates` assumes

$$S \geq 2, \qquad S^2V^2 - (S^2-1)y^2 = 1, \tag{1}$$

with $S, V, y \in \mathbb{Z}$, and proves $y \neq 0$ and $|y| < 2|V|$. Its root-bound tail first argues by contradiction to bound y^2, then argues by contradiction again to pass to absolute values.

The additional observation is the exact identity

$$\boxed{(S^2-1)(4V^2-y^2) = (3S^2-4)V^2 + 1.} \tag{2}$$

To verify it, expand the left-hand side and replace $S^2V^2 - (S^2-1)y^2$ with 1 using (1). Since $S^2 \geq 4$, the right-hand side of (2) is strictly positive. The factor S^2-1 is nonnegative, so $4V^2 - y^2$ must be

positive. Hence

$$y^2 < (2|V|)^2,$$

and the standard absolute-square comparison theorem proves the desired bound.

After the original proof of $y \neq 0$, the replacement is:

```
refine ⟨hy, abs_lt_of_sq_lt_sq ?_ (by positivity)⟩
have hnonneg := mul_nonneg
  (show 0 ≤ 3 * S ^ 2 - 4 by omega) (sq_nonneg V)
have hprod : 0 < (S ^ 2 - 1) * (4 * V ^ 2 - y ^ 2) := by
  nlinarith only [hn, hnonneg]
have hlt := pos_of_mul_pos_right hprod
  (show 0 ≤ S ^ 2 - 1 by omega)
nlinarith only [hlt, sq_abs V]
```

The restricted arithmetic calls have an explicit polynomial certificate behind them. They do not need to discover sign cases for V or y, and the result applies to negative signed ordinates exactly as before.

The statement and the entire original nonzero-y prefix are unchanged. This is deliberate: integer-unit extraction was already proposed elsewhere, so using it here must not be advertised as another new saving. The remaining bound tail is independent of how the nonzero fact is proved. Its declaration decreases from 28 to 23 lines, a five-line incremental reduction.

The scale premise is substantive. At $S = 1, V = 1$, equation (1) holds for every y, so the claimed bound would fail, for example, at $y = 100$. The refactor preserves the scale guard rather than weakening it because the new algebra looks shorter.

3.2 One affine domination theorem serves both directions

The literal carrier equation is

$$y = 1 + 4\big(b(m - V^2G) - G\big). \tag{3}$$

The original soundness and completeness proofs separately establish several estimates involving V^2, bV^2, a quotient witness, and a large coefficient. Their common source becomes visible after setting

$$K = 1 + bV^2.$$

Equation (3) then becomes the affine equation

$$\boxed{4bm = y - 1 + 4KG.} \tag{4}$$

The useful shared theorem is

$$b \in \mathbb{Z}, \quad b > 0 \quad \Longrightarrow \quad 2|V| + 1 < 4(1 + bV^2). \tag{5}$$

It has a direct positive certificate. With $r = |V|$,

$$4(1 + bV^2) - (2r + 1) = (2r - 1)^2 + 2r + 2 + 4(b - 1)V^2. \tag{6}$$

All terms on the right are nonnegative and the constant contribution is positive. Integrality gives $b \geq 1$ from $b > 0$. The implemented helper is four declaration lines:

```
theorem carrier_step_dominates {b : ℤ} (hb : 0 < b) (V : ℤ) :
    2 * |V| + 1 < 4 * (1 + b * V ^ 2) := by
  have h := mul_nonneg (show 0 ≤ b - 1 by omega) (sq_nonneg V)
  nlinarith [sq_abs V, sq_nonneg (2 * |V| - 1), abs_nonneg V]
```

Soundness. In `ThirteenCarrierSoundness.transport_nonneg`, assume $b, m > 0$ and $|y| < 2|V|$. If $G < 0$, then $G \le -1$ because G is an integer. The right-hand side of (4) satisfies

$$y - 1 + 4KG \le y - 1 - 4K < 0$$

by (5), whereas the left-hand side is positive. This contradiction proves $G \ge 0$.

No divisibility premise on G is needed for this direction. Introducing $b \mid G$ to imitate the completeness proof would unnecessarily strengthen the soundness theorem. The candidate retains its original weaker hypotheses.

Completeness. In `ThirteenFreshSeed.carrier_exists`, the hypotheses give

$$G = bg, \qquad y - 1 = 4be$$

for integer witnesses g, e. Keep the existing witness exactly:

$$m = e + (1 + bV^2)g = e + Kg. \tag{7}$$

It satisfies (4) by a linear polynomial identity. Since $G > 0$, integrality gives $G \ge 1$. Since $V > 0$ and $y > -2V$, the right-hand side obeys

$$y - 1 + 4KG \ge y - 1 + 4K > 0.$$

The last inequality uses $4K > 2V + 1$. Therefore $m > 0$. The same normalized equality then closes the literal witness equation, so a second expansion of the quotient witnesses is unnecessary.

This removes a separate proof that $g > 0$ and the chain of coefficient inequalities depending on g. It does not choose a different witness, change the transport coordinate, or require a two-sided bound on y in the completeness theorem. The existing one-sided hypothesis is sufficient.

3.3 Boundary cases and measured cost

The two directions have intentionally different endpoint conditions. With

$$b = m = 1, \quad G = 0, \quad V = 3, \quad y = 5,$$

equation (3) and $|y| < 2|V|$ hold. Thus soundness cannot be strengthened to $G > 0$. Conversely, take

$$b = 1, \quad G = 0, \quad V = 2, \quad y = -3.$$

The completeness hypotheses other than $G > 0$ hold, including $b \mid G$, $4b \mid y - 1$, and $-2V < y$. But the equation forces $m = -1$. Strict positivity of G remains necessary there.

The three declarations change from 28, 14, and 32 lines to 23, 9, and 20 lines. After charging the helper, its documentation, and spacing, the two complete files shrink by 16 lines. Five lines come from the norm-gap tail; the affine changes account for the other eleven. All four exact theorem blocks, including the helper, compiled under the pinned toolchain. The complete carrier modules and their dependency graph were not rebuilt.

4 Replace the helical integer induction by periodicity

The theorem `WangTiles942HelicalQuotient.row_zero_of_helix` assumes

$$T(x+w,t-1)=T(x,t)\qquad (x,t,w\in\mathbb{Z}) \tag{8}$$

and concludes

$$T(x,t)=T(x+wt,0). \tag{9}$$

The existing proof performs separate forward and backward integer induction steps. Both branches repeat the same address arithmetic, with different signs.

For fixed x,t, define

$$f(s)=T(x+ws,t-s).$$

The helical hypothesis says exactly that $f(s+1)=f(s)$. Existing integer-periodicity machinery then gives $f(0)=f(t)$, which is (9). The same argument covers positive and negative times.

```
let f : ℤ → α := fun s => T (x+w*s) (t-s)
have hp : Function.Periodic f 1 := by
  intro s
  dsimp [f]
  convert hhelix (x+w*s) (t-s) using 1; congr 1 <;> ring
simpa [f] using hp.sub_int_mul_eq (x := t) t
```

The mathematical invariant is $x+wt$. Indeed, moving from (x,t) to $(x+w,t-1)$ preserves that quantity. This gives a useful stronger statement, also proved in the supplied kernel:

$$(8)\quad\Longleftrightarrow\quad\exists!F:\mathbb{Z}\to\alpha,\ \forall x,t,\ T(x,t)=F(x+wt). \tag{10}$$

For existence take $F(z)=T(z,0)$. For uniqueness specialize any candidate factorization to $t=0$. The reverse implication follows by the equality $(x+w)+w(t-1)=x+wt$.

The replacement is valid for an arbitrary codomain, arbitrary integer width, and every integer time. It does not assume a positive width, a vertical period, a tiling compatibility condition, or a nonzero finite modulus. Those assumptions belong to later finite-quotient theorems and remain where they are required.

Earlier work on centered interval induction and guarded row propagation concerns extending local information through a bounded spatial interval. Here a global translation law is reduced along its exact orbit. This different remaining obligation is why the proposal survives the earlier induction-related exclusions.

The candidate file changes from 147 to 137 lines. The exact theorem and the stronger unique-factorization result compiled. The full tile-dependent file and subsequent finite-quotient clients were not rebuilt. The stronger result is a demonstrated semantic interface; the production patch only replaces the original theorem body and does not charge an unrequested new public factorization API to the line savings.

5 Use one list-sum bridge for executable monomial semantics

`SyntaxCoefficients.interpret` recursively interprets a list of monomials as an ordinary multivariate polynomial. The executable expansion deliberately retains repeated terms. Its multiplication is a nested `flatMap`/`map`, and coefficient lookup sums all entries with the requested exponent vector.

Five existing list inductions prove append semantics, negation semantics, multiplication by a fixed monomial, general multiplication, and coefficient interpretation. These are different presentations of ordinary additive sum laws.

Introduce a single private, proof-only bridge:

```
private theorem interpret_eq_sum {n : ℕ} (terms : List (Term n)) :
    interpret terms =
      (terms.map fun t => MvPolynomial.monomial (exponent t.1) t.2).sum := by
  induction terms <;> simp_all [interpret]
```

The public proofs can then use the library's exact list operations. Append uses `List.map_append` and `List.sum_append`. Negation uses `List.sum_neg`. Multiplication uses `List.map_flatMap`, `List.flatMap_def`, `List.sum_flatten`, and the two list-sum distributivity laws. The private fixed-monomial induction disappears entirely.

For coefficient interpretation, the essential fact is that a coefficient at a fixed exponent is additive. Use `MvPolynomial.coeffAddMonoidHom` and `map_list_sum`; the monomial coefficient theorem and `exponent_inj` complete the calculation. This is a direct expression of

$$[X^d]\left(\sum_{t\in L} c_t X^{e_t}\right) = \sum_{t\in L} \begin{cases} c_t, & e_t = d, \\ 0, & e_t \neq d. \end{cases}$$

Here $t \in L$ denotes occurrences in the list, including repeated occurrences.

5.1 Why the list representation should remain

Replacing the executable list with a finite set would lose multiplicities unless an additional collection procedure were introduced. Replacing it with a canonical sparse polynomial would also change executable data and could introduce new equality, normalization, and computability obligations. Neither change is necessary for the proof reduction.

The bridge preserves the actual list. If the same monomial occurs twice, both contributions remain; opposite coefficients cancel only in the interpreted sum; zero coefficients remain harmless; and the empty list and zero-variable case need no special exception. The theorem is not marked globally `simp`, so clients retain control of when the recursive interpreter is exposed as a sum.

5.2 Increment and validation

Prior coefficient work concerns finite row catalogs, coefficient envelopes, table naturality, and sparse row-identity checkers. It does not discharge these five monomial-list semantic inductions. The proposed bridge is a small remaining interface between an executable list representation and the existing additive API.

The full file changes from 120 to 111 lines, including the new import and bridge. Five list inductions become one; the syntax induction proving that expansion denotes the original polynomial remains appropriate and is unchanged. The entire replacement module compiled against a freshly compiled pinned `Representation`. The original and replacement `coefficient_expand` axiom sets agree. Other callers of the coefficient module were not rebuilt, so no broader integration claim is made.

6 Eliminate function witnesses that are already determined

After the previously proposed witness-block and assignment normalizations, two composition proofs still have a final goal of the form

$$(\exists a : \iota \to A,\ P(a)\ \wedge\ \forall i,\ a(i) = f(i)) \quad \Longleftrightarrow \quad P(f). \tag{11}$$

The implementation manually destructs the existential, constructs function equality by extensionality, substitutes it, and then reconstructs the canonical function in the reverse direction.

There is a direct proof using only the intended logical normal form:

```
simp only [← funext_iff, exists_eq_right]
```

First turn pointwise equality into equality of functions. Then eliminate the existential whose value is fixed. This is an exact deterministic substitution rule, requiring no choice principle, finite enumeration, or nonempty-domain hypothesis. In particular, it covers the zero-arity function space without an extra case.

The two concrete sites are `ComposeRepresentation.Rep.compose` and `DirectComposition.GraphRep.composeDirect`. Each already ends with a `change` presenting the desired existential shape. The patch changes only the following proof tail. It does not revisit finite witness concatenation, assignment reindexing, arithmetic graph construction, or the earlier representation interfaces.

This distinction also explains what is *not* reduced: the represented equation still has the same stored witnesses and expression syntax. The existential disappears inside the semantic correctness proof, not from the compiled representation. The two full constructors are definitionally equal to the originals, as checked by `rfl` in the compatibility file.

The complete files shrink from 71 to 66 and from 77 to 72 lines. Both were compiled after freshly compiling the pinned `Representation` and `ArithmeticRepresentation`. The exact original constructors were reconstructed, both record-equality checks passed, and their paired axiom reports agree. This is the smallest implementation change in the packet and a suitable first integration step.

7 Validation, accounting, and a practical adoption order

7.1 Evidence has several scopes

The Lean compiler used here reports version 4.32.0, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The checked library sources and repository manifest use mathlib revision `81a5d257c8e410db227a6665ed08f64fea08e997`. Existing compiled dependencies were reused. This was not a rebuild of mathlib from source.

Group	Completed	Still required for integration
Composition	Two full replacements; fresh immediate project dependencies; exact whole-record equalities; paired axiom reports	Rebuild further callers in the target checkout
Monomial lists	Full replacement and original baseline; identical endpoint axiom sets	Rebuild coefficient-extraction clients
Carrier arithmetic	Exact three replacement statements/bodies plus helper; unchanged prefix and witness checks	Full two modules and their callers
Helical theorem	Exact original statement with new body; stronger factorization	Full tile-dependent module and quotient clients
Jones evaluation	Five exact client statements using actual source definitions and shared proofs; full-source comparison	Both complete modules and their callers

The final standalone carrier reports contain only `propext`, `Classical.choice`, and `Quot.sound`. The generic helical and deterministic-witness results use `propext` and `Quot.sound`. The coefficient endpoint has the same standard three-axiom set before and after. The supplied successful checks contain no `sorryAx`. These observations do not substitute for an axiom audit of every endpoint in the full repository.

The independent review compared final source hashes, theorem headers, unchanged definitions, patch output, and kernel bodies. It also checked the mathematical endpoint guards. One initial attempt to save the successful original coefficient-module compiler output encountered a temporary storage error after the compiler had exited successfully; the evidence file explicitly records that its transcript was persisted from the observed tool output. No additional compiler success is inferred from that persistence step.

7.2 Exact file-level accounting

File	Before	After	Removed
`JonesSyntax.lean`	179	179	0
`ExecutableJones.lean`	300	251	49
`ThirteenCarrierSoundness.lean`	290	286	4
`ThirteenFreshSeed.lean`	217	205	12
`WangTiles942HelicalQuotient.lean`	147	137	10
`SyntaxCoefficients.lean`	120	111	9
`ComposeRepresentation.lean`	71	66	5
`DirectComposition.lean`	77	72	5
Total	**1401**	**1307**	**94**

The carrier helper is counted once in `ThirteenCarrierSoundness`; it is not treated as free infrastructure. The unchanged nonzero-ordinate prefix ensures no overlap with the older unit-extraction proposal. The optional stronger helical factorization is included as a checked research artifact, but is not silently added to the production patch while claiming the original ten-line saving.

7.3 Suggested adoption order

Start with the two composition tails: the edits are local and the complete data records have definitional equality evidence. Next adopt the monomial-list bridge and check its direct coefficient clients. Then promote the Jones semantic API and rebuild the executable client; this gives the largest maintenance reduction with no new import edge.

The carrier changes form a coherent arithmetic commit. The shared helper must be added with its two clients, and the original strict/non-strict sign hypotheses should remain visible in review. The helical replacement is independent and can be integrated separately after the tile module rebuild. No change to a polynomial artifact, published equation, or stored program is needed for any group.

The archive's kernel replay script uses an existing repository environment and a temporary output directory for the three complete-module checks. It also replays the isolated kernels and compatibility checks. The source verifier checks file hashes, source-line totals, patch application to the included baseline copies, and final replacement identity. A later full integration build should use the repository's own build and check targets after applying the selected patches on a working branch.

8 Ideas deliberately excluded during this iteration

A non-repetition review should document rejected candidates as well as successful ones. A proposed cleanup of `RatioReparameterization.combined_dvd_iff` initially looked promising because the exact theorem name is absent from prior report prose. Its mechanism, however, is the same as packet l's instruction to keep divisibility witnesses abstract and combine `dvd_add`, `dvd_sub`, and related closure laws rather than unpacking and repacking witnesses. It was dropped and contributes no savings.

A smaller `ExecutableJones.eval_vectorSum` cleanup was also considered. It would save only a few presentation lines around finite-function enumeration and offered less benefit than exposing the already available field-evaluation API. It is not included as an additional headline or as a speculative saving.

The much larger familiar proposals are excluded too: scanner iteration and simulation libraries; finite witness splitting and assignment naturality; shared residual sums; generated circuit certificates; coefficient-envelope tables; signed Pell congruences; integer-unit and rational-root descent; definable internal induction and recursive-graph fusion; automatic depth budgets; cyclic-word quotient interfaces; and the previously proposed Wang controller/row abstractions. The detailed ledger names the relevant packets and preserves the distinction between a general technique already proposed and a genuinely different remaining proof obligation.

9 Archive guide and source references

`report.tex`, `report.pdf`
This complete article and its compiled output.

`patches/`
Four component patches and their combined patch, against the pinned snapshot.

`replacements/`, `baseline/`
The eight complete candidate sources and their corresponding original files, allowing exact offline comparisons.

`lean/`
: Carrier and helical kernels, the literal-source Jones harness, and the composition compatibility checks.

`evidence/`
: Novelty and independent-review notes, compiler transcripts, line-count metrics, source provenance, and source-authentication receipts.

`scripts/`
: Archive verification, Lean replay through an existing project environment, and PDF compilation instructions.

All repository links below are pinned to the reviewed commit. Counts and checking outcomes are original observations recorded in the accompanying evidence files.

[S1] Prior simplification packets a–v, including the adoption record.
[S2] Current incoming index and retirement summaries.
[S3] `JonesSyntax.eval_rad`; `ExecutableJones.eval_withInput`, with their companion field-evaluation lemmas.
[S4] `ThirteenCarrierSoundness.auxiliary_estimates`; `ThirteenCarrierSoundness.transport_nonneg`; `ThirteenFreshSeed.carrier_exists`.
[S5] `WangTiles942HelicalQuotient.row_zero_of_helix`.
[S6] `SyntaxCoefficients.interpret`; `SyntaxCoefficients.coefficient_interpret`.
[S7] `ComposeRepresentation.compose`; `DirectComposition.composeDirect`.
[S8] Mathlib integer periodicity; list-sum distributivity; coefficient additive homomorphism.